A* Algorithm Using Python

Aim

To implement the **A*** search algorithm in Python to find the shortest path from a start node to a goal node.

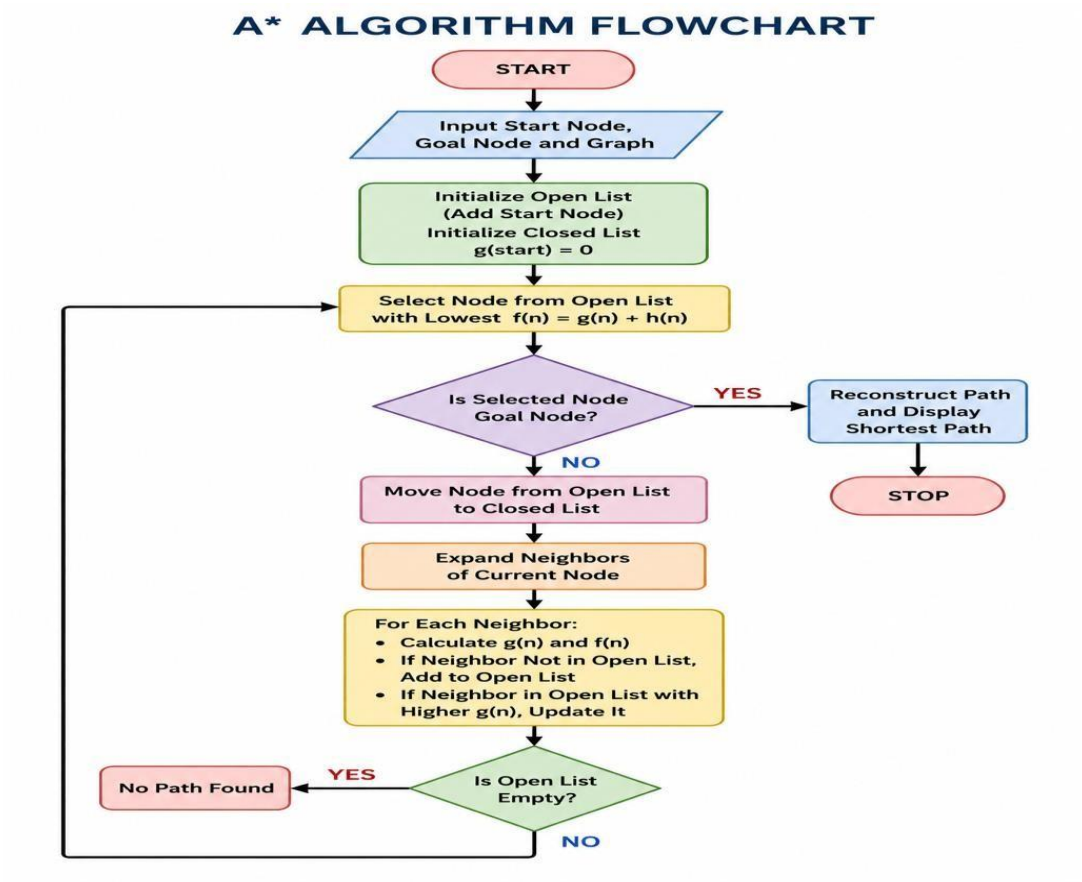
Objective

- To find the optimal path between two nodes.
- To minimize the total path cost.
- To use heuristic values for efficient searching.

Algorithm

1. Initialize the open list with the start node.
2. Initialize the closed list as empty.
3. Select the node with the lowest **f(n) = g(n) + h(n)**.
4. If the selected node is the goal node, stop.
5. Otherwise, expand its neighboring nodes.
6. Calculate f(n) for each neighbor.
7. Add unvisited neighbors to the open list.
8. Repeat until the goal is reached.

FLOWCHART:



CODE:

```
graph = {
    'A': [('B', 1), ('C', 3)],
'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 1)],
    'F': []
}


heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
```

```
    'D': 2,
    'E': 1,
    'F': 0
}

def astar(start, goal):
open_list = [(start, 0)]
closed = set()     g =
{start: 0}     parent =
{start: None}

    while open_list:
        open_list.sort(key=lambda x: x[1])
current = open_list.pop(0)[0]

        if current == goal:
            path = []
while current:
                path.append(current)
current = parent[current]
return path[::-1]

        closed.add(current)

        for neighbor, cost in graph[current]:
new_cost = g[current] + cost

            if neighbor not in g or new_cost < g[neighbor]:
                g[neighbor] = new_cost                f
= new_cost + heuristic[neighbor]
```

open_list.append((neighbor, f))

parent[neighbor] = current

return None

path = astar('A', 'F') print("Shortest

Path:", path) OUTPUT:

Result

The **A*** algorithm was implemented successfully using Python. The algorithm efficiently searched the graph using heuristic values and found the optimal path from the start node to the goal node. The shortest path obtained was **A → B → E → F**.

Conclusion

The **A*** search algorithm successfully found the shortest path by combining the actual path cost **g(n)** and heuristic cost **h(n)**. It is more efficient than uninformed search algorithms and is widely used in pathfinding, robotics, navigation systems, and artificial intelligence applications.